

Converting a Recursive Algorithm into an Iterative Algorithm

Converting a recursive algorithm into an iterative algorithm is always possible. While recursive functions repeat a calculation by calling themselves, this repetition can be performed instead by a loop. Recursive functions also make use of the call stack; however, an iterative algorithm can replace this with a stack data structure. Thus, any recursive algorithm can be performed iteratively by using a loop and a stack.

To demonstrate this, here is *factorialEmulateRecursion.py*, a Python program that implements an iterative algorithm to emulate a recursive algorithm:

```
callStack = [] # The explicit call stack, which holds "frame objects". ❶
callStack.append({'returnAddr': 'start', 'number': 5}) # "Call" the "factorial()
function". ❷
returnValue = None

while len(callStack) > 0:
    # The body of the "factorial() function":

    number = callStack[-1]['number'] # Set number parameter.
    returnAddr = callStack[-1]['returnAddr']

    if returnAddr == 'start':
        if number == 1:
            # BASE CASE
            returnValue = 1
            callStack.pop() # "Return" from "function call". ❸
            continue
        else:
            # RECURSIVE CASE
            callStack[-1]['returnAddr'] = 'after recursive call'
            # "Call" the "factorial() function":
            callStack.append({'returnAddr': 'start', 'number': number - 1}) ❹
            continue
    elif returnAddr == 'after recursive call':
        returnValue = number * returnValue
        callStack.pop() # "Return from function call". ❺
```